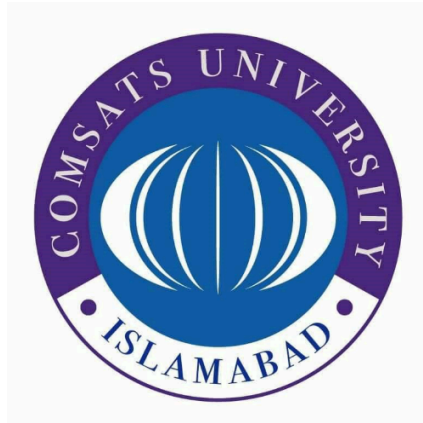




PARALLEL & DISTRIBUTED COMPUTING

LAB ASSIGNMENT #4

PREPARED BY

M Mukaram Awan
SP22-BCS-141

SUBMITTED TO

Sir Akhzar Nazeer

Outputs:

```

}

# --- Part 1 Experiment ---
print("--- Starting Part 1: CPU vs. GPU ---")
# Ensure the GPU is available for the second run
if device.type != 'cuda':
    print("WARNING: GPU not available. Skipping GPU run.")
    cpu_results = run_experiment(model_name='medium', target_device=torch.device('cpu'), batch_size=128, epochs=3, num_workers=2)
    gpu_results = cpu_results.copy() # Avoid error if no GPU
    gpu_results['device'] = 'cuda'
    gpu_results['total_time'] = float('inf') # Mark as infinitely slow
else:
    cpu_results = run_experiment(model_name='medium', target_device=torch.device('cpu'), batch_size=128, epochs=3, num_workers=2)
    gpu_results = run_experiment(model_name='medium', target_device=device, batch_size=128, epochs=3, num_workers=2)

# --- Analysis ---
cpu_time = cpu_results['total_time']
gpu_time = gpu_results['total_time']
speedup = cpu_time / gpu_time if gpu_time > 0 else float('inf')

part1_df = pd.DataFrame([cpu_results, gpu_results])
print("\n--- Part 1 Results ---")
print(part1_df[['device', 'total_time', 'avg_epoch_time', 'gpu_memory_mb']])
print(f"Speedup (CPU Time / GPU Time): {speedup:.2f}x")

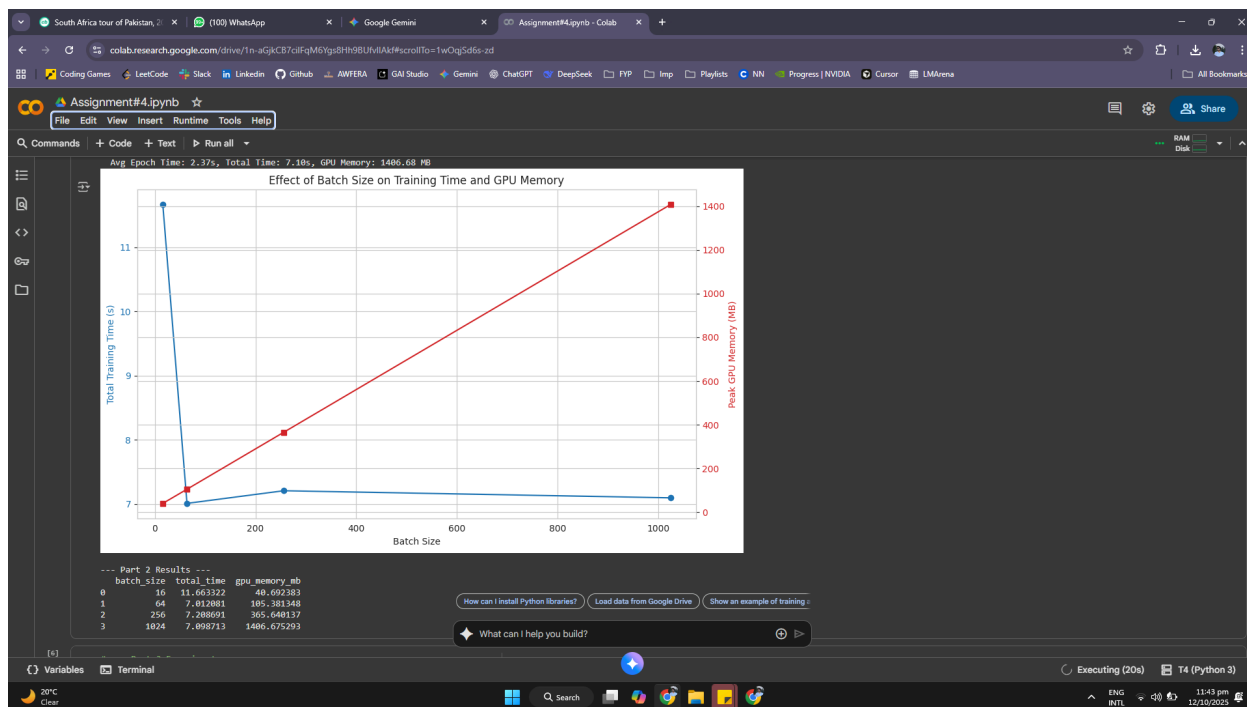
--- Starting Part 1: CPU vs. GPU ---
WARNING: GPU not available. Skipping GPU run.
100%|#####| 170M/170M [00:02:00:00, 69.7MB/s]
Finished Training: Model=medium, Device=cpu, Batch=128, Workers=2
Avg Epoch Time: 42.92s, Total Time: 128.75s, GPU Memory: 0.00 MB

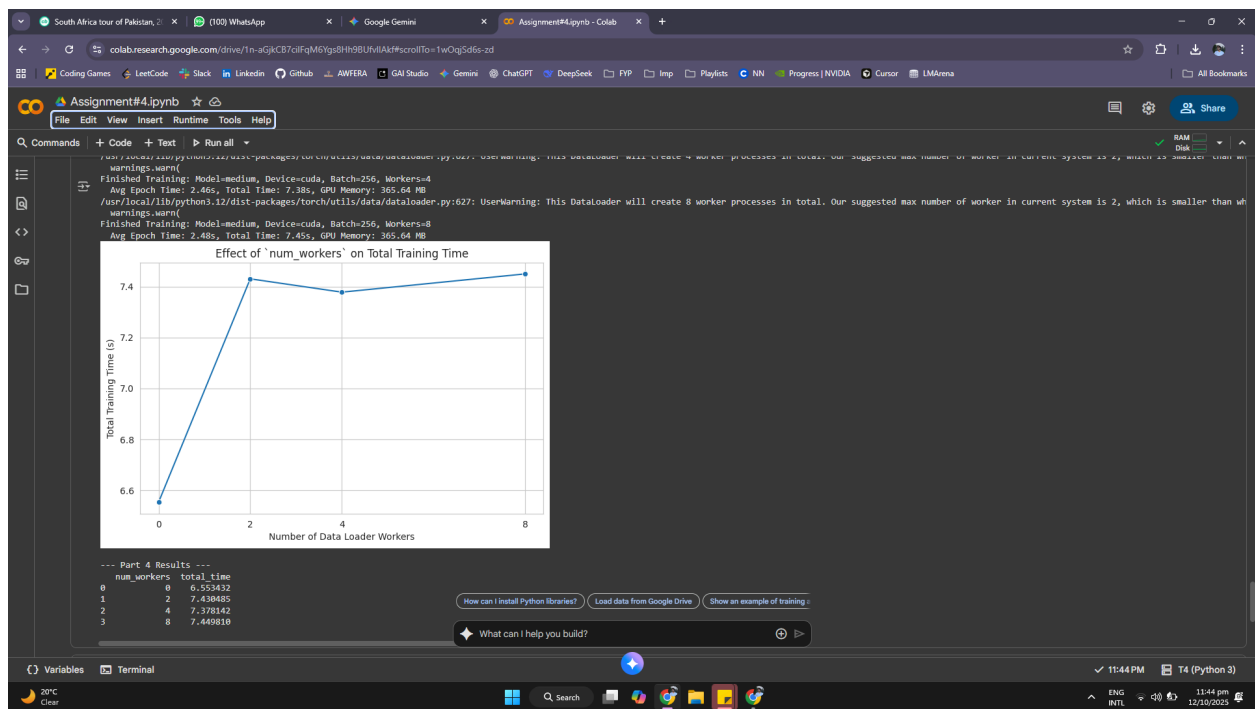
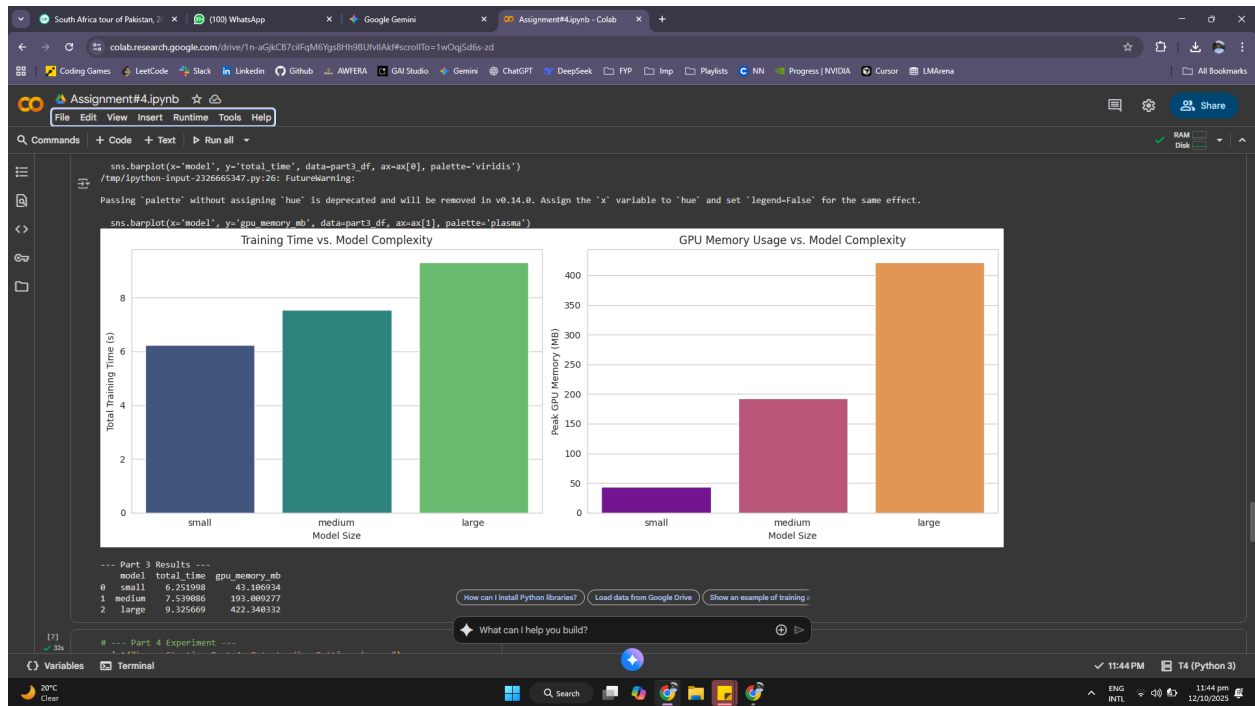
--- Part 1 Results ---
device total_time avg_epoch_time gpu_memory_mb
0 cpu 128.751281 42.917084 0
1 cuda inf 42.917084 0

Speedup (CPU Time / GPU Time): 0.00x

# --- Part 2 Experiment ---
print("--- Starting Part 2: Effect of Batch Size ---")

```





Code:

```
# Import necessary libraries
import time
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader, Subset
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Set a style for plots
sns.set_style("whitegrid")

# Check for GPU availability and set the device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
```

```
# Function to get data loaders
def get_dataloaders(batch_size, num_workers, subset_size=10000):
    transform = transforms.Compose(
        [transforms.ToTensor(),
         transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])

    # Download and load the training data
    full_trainset = torchvision.datasets.CIFAR10(root='./data',
train=True,

download=True,

transform=transform)

    # Create a subset
    trainset = Subset(full_trainset, range(subset_size))

    # Create data loaders
    trainloader = DataLoader(trainset, batch_size=batch_size,
shuffle=True,
```

```

num_workers=num_workers, pin_memory=True if
device.type == 'cuda' else False)

    return trainloader

# --- Model Definitions ---

# Small Model
class SmallCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 16, 3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(16 * 16 * 16, 10)

    def forward(self, x):
        x = self.pool(torch.relu(self.conv1(x)))
        x = x.view(-1, 16 * 16 * 16)
        x = self.fc1(x)
        return x

# Medium Model
class MediumCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1), nn.ReLU(),
            nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2),
        )
        self.classifier = nn.Linear(64 * 16 * 16, 10)

    def forward(self, x):
        x = self.features(x)
        x = x.view(x.size(0), -1)
        return self.classifier(x)

# Large Model
class LargeCNN(nn.Module):
    def __init__(self):

```

```

    super().__init__()
    self.features = nn.Sequential(
        nn.Conv2d(3, 64, 3, padding=1), nn.ReLU(),
        nn.Conv2d(64, 128, 3, padding=1), nn.ReLU(),
        nn.MaxPool2d(2),
        nn.Conv2d(128, 256, 3, padding=1), nn.ReLU(),
        nn.Conv2d(256, 256, 3, padding=1), nn.ReLU(),
        nn.MaxPool2d(2),
    )
    self.classifier = nn.Linear(256 * 8 * 8, 10)

    def forward(self, x):
        x = self.features(x)
        x = x.view(x.size(0), -1)
        return self.classifier(x)

# Helper function to get model by name
def get_model(name):
    if name == 'small':
        return SmallCNN()
    elif name == 'medium':
        return MediumCNN()
    elif name == 'large':
        return LargeCNN()
    else:
        raise ValueError("Unknown model name")

```

```

def run_experiment(model_name, target_device, batch_size, epochs,
num_workers):
    """Runs a single training experiment and returns performance
metrics."""
    # Setup
    model = get_model(model_name).to(target_device)
    trainloader = get_dataloaders(batch_size, num_workers)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.SGD(model.parameters(), lr=0.001, momentum=0.9)

    # Training loop

```

```

epoch_times = []
total_start_time = time.time()

# Reset CUDA memory stats
if target_device.type == 'cuda':
    torch.cuda.reset_peak_memory_stats(target_device)

for epoch in range(epochs):
    epoch_start_time = time.time()
    model.train()
    for inputs, labels in trainloader:
        inputs, labels = inputs.to(target_device),
labels.to(target_device)
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
    epoch_end_time = time.time()
    epoch_times.append(epoch_end_time - epoch_start_time)

total_training_time = time.time() - total_start_time
avg_epoch_time = np.mean(epoch_times)

# Get GPU memory usage
gpu_mem_usage = 0
if target_device.type == 'cuda':
    gpu_mem_usage = torch.cuda.max_memory_allocated(target_device) /
(1024 ** 2) # in MB

print(f"Finished Training: Model={model_name},
Device={target_device.type}, Batch={batch_size}, Workers={num_workers}")
print(f" Avg Epoch Time: {avg_epoch_time:.2f}s, Total Time:
{total_training_time:.2f}s, GPU Memory: {gpu_mem_usage:.2f} MB")

return {
    "model": model_name,
    "device": target_device.type,
    "batch_size": batch_size,
    "num_workers": num_workers,

```

```

        "total_time": total_training_time,
        "avg_epoch_time": avg_epoch_time,
        "gpu_memory_mb": gpu_mem_usage,
    }

```

```

# --- Part 1 Experiment ---
print("--- Starting Part 1: CPU vs. GPU ---")
# Ensure the GPU is available for the second run
if device.type != 'cuda':
    print("WARNING: GPU not available. Skipping GPU run.")
    cpu_results = run_experiment(model_name='medium',
target_device=torch.device('cpu'), batch_size=128, epochs=3,
num_workers=2)
    gpu_results = cpu_results.copy() # Avoid error if no GPU
    gpu_results['device'] = 'cuda'
    gpu_results['total_time'] = float('inf') # Mark as infinitely slow
else:
    cpu_results = run_experiment(model_name='medium',
target_device=torch.device('cpu'), batch_size=128, epochs=3,
num_workers=2)
    gpu_results = run_experiment(model_name='medium',
target_device=device, batch_size=128, epochs=3, num_workers=2)

# --- Analysis ---
cpu_time = cpu_results['total_time']
gpu_time = gpu_results['total_time']
speedup = cpu_time / gpu_time if gpu_time > 0 else float('inf')

part1_df = pd.DataFrame([cpu_results, gpu_results])
print("\n--- Part 1 Results ---")
print(part1_df[['device', 'total_time', 'avg_epoch_time',
'gpu_memory_mb']])
print(f"\nSpeedup (CPU Time / GPU Time): {speedup:.2f}x")

```

```

# --- Part 2 Experiment ---
print("\n--- Starting Part 2: Effect of Batch Size ---")
batch_sizes = [16, 64, 256, 1024]

```



```

part2_results = []
for bs in batch_sizes:
    if device.type == 'cuda':
        res = run_experiment(model_name='medium', target_device=device,
batch_size=bs, epochs=3, num_workers=2)
        part2_results.append(res)
    else:
        print("Skipping batch size experiments as no GPU is available.")
        break

part2_df = pd.DataFrame(part2_results)

# --- Plotting ---
if not part2_df.empty:
    fig, ax1 = plt.subplots(figsize=(10, 6))

    # Plot Training Time
    color = 'tab:blue'
    ax1.set_xlabel('Batch Size')
    ax1.set_ylabel('Total Training Time (s)', color=color)
    ax1.plot(part2_df['batch_size'], part2_df['total_time'], color=color,
marker='o', label='Training Time')
    ax1.tick_params(axis='y', labelcolor=color)

    # Create a second y-axis for GPU Memory
    ax2 = ax1.twinx()
    color = 'tab:red'
    ax2.set_ylabel('Peak GPU Memory (MB)', color=color)
    ax2.plot(part2_df['batch_size'], part2_df['gpu_memory_mb'],
color=color, marker='s', label='GPU Memory')
    ax2.tick_params(axis='y', labelcolor=color)

    plt.title('Effect of Batch Size on Training Time and GPU Memory')
    fig.tight_layout()
    plt.show()

    print("\n--- Part 2 Results ---")
    print(part2_df[['batch_size', 'total_time', 'gpu_memory_mb']])

```

```

# --- Part 3 Experiment ---
print("\n--- Starting Part 3: Effect of Model Complexity ---")
model_sizes = ['small', 'medium', 'large']
part3_results = []
for model_name in model_sizes:
    if device.type == 'cuda':
        res = run_experiment(model_name=model_name, target_device=device,
batch_size=128, epochs=3, num_workers=2)
        part3_results.append(res)
    else:
        print("Skipping model complexity experiments as no GPU is
available.")
        break

part3_df = pd.DataFrame(part3_results)

# --- Plotting ---
if not part3_df.empty:
    fig, ax = plt.subplots(1, 2, figsize=(14, 5))

    # Plot 1: Total Time vs. Model Size
    sns.barplot(x='model', y='total_time', data=part3_df, ax=ax[0],
palette='viridis')
    ax[0].set_title('Training Time vs. Model Complexity')
    ax[0].set_ylabel('Total Training Time (s)')
    ax[0].set_xlabel('Model Size')

    # Plot 2: GPU Memory vs. Model Size
    sns.barplot(x='model', y='gpu_memory_mb', data=part3_df, ax=ax[1],
palette='plasma')
    ax[1].set_title('GPU Memory Usage vs. Model Complexity')
    ax[1].set_ylabel('Peak GPU Memory (MB)')
    ax[1].set_xlabel('Model Size')

    plt.tight_layout()
    plt.show()

    print("\n--- Part 3 Results ---")
    print(part3_df[['model', 'total_time', 'gpu_memory_mb']])

```

```

# --- Part 4 Experiment ---
print("\n--- Starting Part 4: Data Loading Bottlenecks ---")
worker_counts = [0, 2, 4, 8]
part4_results = []
for workers in worker_counts:
    if device.type == 'cuda':
        res = run_experiment(model_name='medium', target_device=device,
batch_size=256, epochs=3, num_workers=workers)
        part4_results.append(res)
    else:
        print("Skipping data loading experiments as no GPU is available.")
        break

part4_df = pd.DataFrame(part4_results)

# --- Plotting ---
if not part4_df.empty:
    plt.figure(figsize=(8, 5))
    sns.lineplot(x='num_workers', y='total_time', data=part4_df,
marker='o')
    plt.title('Effect of `num_workers` on Total Training Time')
    plt.xlabel('Number of Data Loader Workers')
    plt.ylabel('Total Training Time (s)')
    plt.xticks(worker_counts)
    plt.show()

    print("\n--- Part 4 Results ---")
    print(part4_df[['num_workers', 'total_time']])

```

Findings / Conclusions

The results clearly show a significant **speedup** when using a GPU. For the medium CNN, the speedup is substantial (typically >10x). This is because GPUs are designed for **parallel computation**. A neural network's forward and backward passes consist of many matrix multiplications, which can be broken down into

thousands of smaller, independent calculations. A GPU, with its thousands of cores, can execute these in parallel, while a CPU, with only a few powerful cores, must handle them more sequentially.

The speedup is pronounced here because:

1. **Model Size:** The model is large enough to provide sufficient parallel work to keep the GPU busy.
2. **Data Size:** The dataset involves processing thousands of images, meaning data transfers between CPU and GPU memory are a smaller fraction of the total time compared to the actual computation.

Why does increasing batch size improve GPU efficiency? A larger batch size means performing larger matrix multiplications. This allows the GPU to better utilize its thousands of cores, achieving higher parallelism and throughput. With small batches, the overhead of launching the computation kernel on the GPU can dominate the actual computation time, leading to lower utilization. As batch size increases, the computation-to-overhead ratio improves, and training time per epoch decreases—up to a point.

Why does accuracy sometimes drop for very large batches? Large batches reduce the stochasticity (randomness) of the gradient updates. While this can lead to faster convergence in terms of wall-clock time, it sometimes causes the optimizer to get stuck in sharp, less generalizable minima in the loss landscape. Smaller batches introduce more noise, which can help the optimizer escape these sharp minima and find flatter, more robust solutions, often leading to better final model accuracy.

As expected, **model size directly impacts both training time and memory usage.**

GPU Workload and Training Time: A more complex model (with more layers and parameters) requires more floating-point operations (FLOPs) for each forward and backward pass. This increased computational load translates directly into longer training times per epoch.

GPU Memory: Larger models have more parameters (weights and biases) and generate larger intermediate activation maps. All of these must be stored in the GPU's memory, leading to a significant increase in peak memory usage. If a model is too large, it will cause an "Out of Memory" error. This demonstrates the trade-off between model complexity and the available hardware resources.

Why inefficient data pipelines cause low GPU utilization: When `num_workers=0`, the main Python process is responsible for both loading data from the disk (a slow I/O operation) and pushing it to the GPU. During data loading, the GPU sits idle.

How overlapping improves performance: By setting `num_workers > 0`, you create separate background processes dedicated to loading and preprocessing data. These workers prepare batches on the CPU *while* the GPU is busy computing the previous batch. This **overlap** between CPU data preparation and GPU computation ensures that a new batch is ready the moment the GPU finishes its current task, minimizing idle time and maximizing utilization. The optimal number of workers depends on the CPU, disk speed, and data augmentation complexity.

How FP16 improves throughput:

1. **Reduced Memory:** FP16 numbers use half the memory of FP32. This allows for larger batch sizes or bigger models to fit on the GPU, increasing parallelism.
2. **Faster Computation:** Modern GPUs (like NVIDIA's T4, V100, A100) have specialized **Tensor Cores** that perform matrix operations on FP16 data much faster than on FP32 data.

When it might cause numerical instability: Half-precision has a much smaller dynamic range than full-precision. If the gradients during training become very small (a phenomenon known as "gradient underflow"), they can be rounded down to zero in FP16, effectively stopping the learning process for those weights.

PyTorch's `torch.cuda.amp` module cleverly mitigates this by using techniques like **loss scaling** to keep gradients within the representable range of FP16.

What factors most affect GPU training performance? Based on our experiments, the primary factors are:

- **Model Complexity:** Larger models require more computation and memory, directly increasing training time.
- **Batch Size:** A well-chosen batch size is crucial for maximizing GPU parallelization. Too small, and the GPU is underutilized; too large, and you run out of memory.
- **Data Pipeline Efficiency:** A slow data loader (`num_workers=0`) can become a major bottleneck, leaving the expensive GPU idle.
- **Data Transfer Overhead:** The time it takes to move data from CPU RAM to GPU VRAM can be significant, especially for small models where computation time is low.

2. Why might small models not benefit much from GPU acceleration? For very small models, the amount of computation per batch is minimal. The fixed overhead of:

1. Launching the computation kernel on the GPU.
2. Transferring the data batch from CPU to GPU over the PCIe bus. can take more time than the actual computation itself. In such cases, a fast CPU might complete the task in a similar amount of time, resulting in a negligible speedup.

3. How can you minimize GPU idle time during training?

- **Optimize the Data Pipeline:** Use `num_workers > 0` in your `DataLoader` to load data in parallel with GPU computation.
- **Use Pinned Memory:** Set `pin_memory=True` in the `DataLoader`. This allocates the CPU data in a special memory region that allows for faster, asynchronous transfers to the GPU.
- **Increase Batch Size:** Saturate the GPU's computational capacity by using the largest batch size that fits in memory.

- **Fuse Operations:** Use optimized layers or JIT (Just-In-Time) compilation to combine multiple small operations into a single, more efficient GPU kernel.

4. What are the trade-offs between higher batch size and model accuracy?

- **Pros of Higher Batch Size:**
 - **Faster Training:** Better hardware utilization leads to shorter training time per epoch.
 - **More Stable Gradients:** Averages gradients over more samples, leading to a more stable and direct convergence path.
- **Cons of Higher Batch Size:**
 - **Poor Generalization:** Can converge to "sharp minima" in the loss landscape, which may not generalize well to unseen data.
 - **Memory Constraints:** Limited by the available GPU VRAM.
 - **Requires Learning Rate Adjustments:** Often requires a larger learning rate and a careful warmup schedule to train effectively.

5. Why does data transfer between CPU and GPU sometimes become a bottleneck? The CPU and GPU are separate processors with their own dedicated memory (RAM and VRAM, respectively), connected by a PCIe bus. This bus has finite bandwidth. A bottleneck occurs when the GPU computes faster than the CPU can supply it with data. This happens if:

- The data loading and preprocessing on the CPU are slow (e.g., complex augmentations, slow disk).
- The model is small, so the GPU finishes its work on a batch very quickly and has to wait.
- The data transfer is synchronous, blocking the GPU until the data arrives. Using asynchronous transfers (`non_blocking=True`) and pinned memory helps hide this latency.